Switch Speeds and Forwarding Methods

7.4.1

Frame Forwarding Methods on Cisco Switches

As you learned in the previous topic, switches use their MAC address tables to determine which port to use to forward frames. With Cisco switches, there are actually two frame forwarding methods and there are good reasons to use one instead of the other, depending on the situation.

Switches use one of the following forwarding methods for switching data between network ports:

- **Store-and-forward switching** – This frame forwarding method receives the entire frame and computes the CRC. CRC uses a mathematical formula, based on the number of bits (1s) in the frame, to determine whether the received frame has an error. If the CRC is valid, the switch looks up the destination address, which determines the outgoing interface. Then the frame is forwarded out of the correct port.
- **Cut-through switching** – This frame forwarding method forwards the frame before it is entirely received. At a minimum, the destination address of the frame must be read before the frame can be forwarded.

A big advantage of store-and-forward switching is that it determines if a frame has errors before propagating the frame. When an error is detected in a frame, the switch discards the frame. Discarding frames with errors reduces the amount of bandwidth consumed by corrupt data. Store-and-forward switching is required for quality of service (QoS) analysis on converged networks where frame classification for traffic prioritization is necessary. For example, voice over IP (VoIP) data streams need to have priority over web-browsing traffic.

Click Play in the animation for a demonstration of the store-and-forward process.

7.4.2

Cut-Through Switching

In cut-through switching, the switch acts upon the data as soon as it is received, even if the transmission is not complete. The switch buffers just enough of the frame to read the destination MAC address so that it can determine to which port it should forward out the data. The destination MAC address is located in the first 6 bytes of the frame following the preamble. The switch looks up the destination MAC address in its switching table, determines the outgoing interface port, and forwards the frame onto its destination through the designated switch port. The switch does not perform any error checking on the frame.

Click Play in the animation for a demonstration of the cut-through switching process.

There are two variants of cut-through switching:

- **Fast-forward switching** – Fast-forward switching offers the lowest level of latency. Fast-forward switching immediately forwards a packet after reading the destination address. Because fast-forward switching starts forwarding before the entire packet has been received, there may be times when packets are relayed with errors. This occurs infrequently, and the destination NIC discards the faulty packet upon receipt. In fast-forward mode, latency is measured from the first bit received to the first bit transmitted. Fast-forward switching is the typical cut-through method of switching.
- **Fragment-free switching** – In fragment-free switching, the switch stores the first 64 bytes of the frame before forwarding. Fragment-free switching can be viewed as a compromise between store-and-forward switching and fast-forward switching. The reason fragment-free switching stores only the first 64 bytes of the frame is that most network errors and collisions occur during the first 64 bytes. Fragment-free switching tries to enhance fast-forward switching by performing a small error check on the first 64 bytes of the frame to ensure that a collision has not occurred before forwarding the frame. Fragment-free switching is a compromise between the high latency and high integrity of store-and-forward switching, and the low latency and reduced integrity of fast-forward switching.

Some switches are configured to perform cut-through switching on a per-port basis until a user-defined error threshold is reached, and then they automatically change to store-and-forward. When the error rate falls below the threshold, the port automatically changes back to cut-through switching.

7.4.3

Memory Buffering on Switches

An Ethernet switch may use a buffering technique to store frames before forwarding them. Buffering may also be used when the destination port is busy because of congestion. The switch stores the frame until it can be transmitted.

As shown in the table, there are two methods of memory buffering:

Memory Buffering Methods

Method	Description
Port-based memory	• Frames are stored in queues that are linked to specific incoming and outgoing ports. • A frame is transmitted to the outgoing port only when all the frames ahead in the queue have been successfully transmitted. • It is possible for a single frame to delay the transmission of all the frames in memory because of a busy destination port. • This delay occurs even if the other frames could be transmitted to open destination ports.
Shared memory	• Deposits all frames into a common memory buffer shared by all switch ports and the amount of buffer memory required by a port is dynamically allocated. • The frames in the buffer are dynamically linked to the destination port enabling a packet to be received on one port and then transmitted on another port, without moving it to a different queue.

Shared memory buffering also results in the ability to store larger frames with potentially fewer dropped frames. This is important with asymmetric switching which allows for different data rates on different ports such as when connecting a server to a 10 Gbps switch port and PCs to 1 Gbps ports.

7.4.4

Duplex and Speed Settings

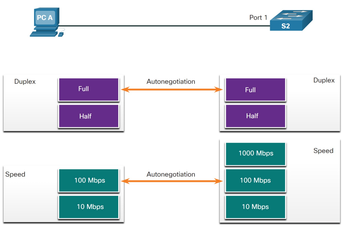
Two of the most basic settings on a switch are the bandwidth (sometimes referred to as "speed") and duplex settings for each individual switch port. It is critical that the duplex and bandwidth settings match between the switch port and the connected devices, such as a computer or another switch.

There are two types of duplex settings used for communications on an Ethernet network:

- **Full-duplex** – Both ends of the connection can send and receive simultaneously.
- **Half-duplex** – Only one end of the connection can send at a time.

Autonegotiation is an optional function found on most Ethernet switches and NICs. It enables two devices to automatically negotiate the best speed and duplex capabilities. Full-duplex is chosen if both devices have the capability along with their highest common bandwidth.

In the figure, the Ethernet NIC for PC-A can operate in full-duplex or half-duplex, and in 10 Mbps or 100 Mbps.



PC-A is connected to switch S2 on port 1, which can operate in full-duplex or half-duplex, and in 10 Mbps, 100 Mbps or 1000 Mbps (1 Gbps). If both devices are using autonegotiation, the operating mode will be full-duplex and 100 Mbps.

Note: Most Cisco switches and Ethernet NICs default to autonegotiation for speed and duplex. Gigabit Ethernet ports only operate in full-duplex.

Duplex mismatch is one of the most common causes of performance issues on 10/100 Mbps Ethernet links. It occurs when one port on the link operates at half-duplex while the other port operates at full-duplex, as shown in the figure.

S2 will continually experience collisions because S1 keeps sending frames any time it has something to send.

Duplex mismatch occurs when one or both ports on a link are reset, and the autonegotiation process does not result in both link partners having the same configuration. It also can occur when users reconfigure one side of a link and forget to reconfigure the other. Both sides of a link should have autonegotiation on, or both sides should have it off. Best practice is to configure both Ethernet switch ports as full-duplex.

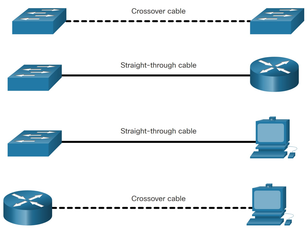
7.4.5

Auto-MDIX

Connections between devices once required the use of either a crossover or straight-through cable. The type of cable required depended on the type of interconnecting devices.

For example, the figure identifies the correct cable type required to interconnect switch-to-switch, switch-to-router, switch-to-host, or router-to-host devices. A crossover cable is used when connecting like devices, and a straight-through cable is used for connecting unlike devices.

Note: A direct connection between a router and a host requires a cross-over connection.



Most switch devices now support the automatic medium-dependent interface crossover (auto-MDIX) feature. When enabled, the switch automatically detects the type of cable attached to the port and configures the interfaces accordingly. Therefore, you can use either a crossover or a straight-through cable for connections to a copper 10/100/1000 port on the switch, regardless of the type of device on the other end of the connection.

The auto-MDIX feature is enabled by default on switches running Cisco IOS Release 12.2(18)SE or later. However, the feature could be disabled. For this reason, you should always use the correct cable type and not rely on the auto-MDIX feature. Auto-MDIX can be re-enabled using the **mdix auto** interface configuration command.

7.4.6

Check Your Understanding – Switch Speeds and Forwarding Methods

Check your understanding of switch speeds and forwarding methods by choosing the correct answer to the following questions.

1. What are two methods for switching data between ports on a switch? (Choose two.)
   - [ ] cut-off switching
   - [ ] cut-through switching
   - [ ] store-and-forward switching
   - [ ] store-and-supply switching
   - [ ] store-and-restore switching
2. Which switching method can be implemented using fast-forward switching or fragment-free switching?
   - ( ) cut-off switching
   - ( ) cut-through switching
   - ( ) store-and-forward switching
   - ( ) store-and-restore switching
3. Which two types of memory buffering techniques are used by switches? (Choose two.)
   - [ ] long-term memory buffering
   - [ ] port-based memory buffering
   - [ ] shared memory buffering
   - [ ] short-term memory buffering
4. What feature automatically negotiates the best speed and duplex setting between interconnecting devices?
   - ( ) auto-MDIX
   - ( ) autoboot
   - ( ) autonegotiation
   - ( ) autotune

Check

Show Me

Reset